

Arduino Walkthrough: Your First Sketch

Programming — Medina County Career Center

What We're Doing

Today you're going to set up the Arduino IDE, connect your Arduino Uno R4 WiFi board, and write your first program (called a "sketch" in Arduino). By the end of this walkthrough, you'll have an LED blinking on your board.

Part 1 — Open the Arduino IDE (Install at home if you'd like)

We have the IDE and the board preinstalled on our machines at MCCC, but if you need to download the IDE and install the board (maybe from home), you can follow these instructions:

Download the IDE if needed

1. Open a browser and search for "**Arduino IDE download**"
2. Go to <https://www.arduino.cc/en/software>
3. Download **Arduino IDE 2.x** for your operating system (Windows, Mac, or Linux)
4. Run the installer and follow the prompts (accept the license, install for your user)

When it's done, open the Arduino IDE. You should see a window with a code editor and a toolbar at the top.

Install the Board Package

The IDE needs to know about your specific board:

1. Click the **Board Manager** icon on the left sidebar (looks like a circuit board)
2. In the search box, type: **Arduino UNO R4**
3. Find **Arduino UNO R4 Boards** and click **Install**
4. Wait for the installation to complete — you'll see progress in the bottom panel

This installs the drivers and tools needed to compile code for the R4 WiFi.

Part 2 — Connect Your Board

Plug In the Arduino

1. Find your Arduino Uno R4 WiFi and the USB-C cable from your kit
2. Plug the USB-C cable into the Arduino and the other end into your computer
3. You should hear your computer's "device connected" sound

Select the Board and Port

1. Go to **Tools → Board → Arduino UNO R4 Boards → Arduino UNO R4 WiFi**

2. Go to **Tools → Port** and select the port that shows **Arduino UNO R4 WiFi**

- On Windows, it will look like **COM5** or **COM7**
- On Mac, it will look like **/dev/cu.usbmodem...**

Tip: If you ever get upload errors, the first thing to check is that the correct board and port are selected.

Part 3 — The Bare Minimum Sketch

Go to **File → Examples → 01.Basics → BareMinimum** to see the simplest possible Arduino program:

```
void setup() {  
    // put your setup code here, to run once:  
}  
  
void loop() {  
    // put your loop code here, to run repeatedly:  
}
```

What to notice:

- **setup()** runs **once** when the board powers on or resets
- **loop()** runs **over and over** forever after setup finishes
- This is different from C where everything starts in **main()**
- Comments use **//** just like in C

Click the **Verify** button (checkmark) to compile it. You should see "Done compiling" at the bottom. This program doesn't do anything yet — but it compiles!

Part 4 — Turn On the LED

Now let's do something. Create a new sketch: **File → New Sketch**

Delete whatever is there and type this:

```
void setup() {  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(13, HIGH);  
}
```

What this does:

- **pinMode(13, OUTPUT)** — tells the Arduino that pin 13 will be used to send signals
- **digitalWrite(13, HIGH)** — sends 5 volts to pin 13, turning the built-in LED on

Upload It

1. Click the **Upload** button (right arrow)
2. Wait for "Done uploading" in the status bar
3. Look at your board — the small LED near pin 13 should be **on** (solid, not blinking)

You just ran your first Arduino program.

Turn It Off

Change **HIGH** to **LOW**:

```
void loop() {  
    digitalWrite(13, LOW);  
}
```

Upload again. The LED turns off. You now control hardware with code.

Part 5 — Make It Blink

Here's where it gets fun. We want the LED to blink: on, off, on, off.

Type this sketch:

```
void setup() {  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(13, HIGH);    // Turn LED on  
    delay(1000);               // Wait 1 second  
    digitalWrite(13, LOW);    // Turn LED off  
    delay(1000);               // Wait 1 second  
}
```

Upload it. **The LED should blink: 1 second on, 1 second off.**

Why Do We Need delay()?

Try removing the delays:

```
void loop() {  
    digitalWrite(13, HIGH);  
    digitalWrite(13, LOW);  
}
```

Upload this. What happens? The LED looks dim instead of blinking. That's because the Arduino runs these two lines millions of times per second — your eyes can't see the individual blinks. The `delay()` slows things down so you can see the on/off cycle.

Experiment with Timing

Try changing the delay values:

```
digitalWrite(13, HIGH);  
delay(100);           // On for 0.1 seconds  
digitalWrite(13, LOW);  
delay(100);           // Off for 0.1 seconds
```

Upload and observe. The LED blinks much faster. Try `50`, `25`, `10` — at what point can you no longer see the blink?

Try **asymmetric** timing:

```
digitalWrite(13, HIGH);  
delay(1000);           // On for 1 second  
digitalWrite(13, LOW);  
delay(200);            // Off for 0.2 seconds
```

This makes a "long on, short off" pattern — like a heartbeat.

Part 6 — Add Serial Output

The **Serial Monitor** lets your Arduino send text back to your computer. This is incredibly useful for debugging.

Type this sketch:

```
void setup() {  
  pinMode(13, OUTPUT);  
  Serial.begin(9600);  
  Serial.println("Arduino is ready!");  
}  
  
void loop() {  
  Serial.println("LED ON");  
  digitalWrite(13, HIGH);  
  delay(1000);  
  
  Serial.println("LED OFF");  
  digitalWrite(13, LOW);  
  delay(1000);  
}
```

Upload it, then open the **Serial Monitor** (Tools → Serial Monitor, or click the magnifying glass icon in the top right). Make sure the baud rate at the bottom is set to **9600**.

You should see:

```
Arduino is ready!  
LED ON  
LED OFF  
LED ON  
LED OFF  
...
```

What to notice:

- `Serial.begin(9600)` starts the serial connection (do this in `setup()`)
- `Serial.println()` prints a line of text (like `printf` in C, but simpler)
- **9600** is the **baud rate** — the speed of communication. Both sides must match.
- The Serial Monitor is your `printf` for Arduino — use it to debug!

Completion Checklist

- ☐ Arduino IDE installed and board package installed
- ☐ Arduino connected via USB — correct board and port selected
- ☐ LED turned on (solid) with `digitalWrite(13, HIGH)`
- ☐ LED turned off with `digitalWrite(13, LOW)`
- ☐ LED blinking with `delay()` controlling the timing
- ☐ Serial Monitor showing output from `Serial.println()`

When all boxes are checked, show your screen to your instructor. You're done!